

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 3 – KONSEP DASAR PEMROGRAMAN
BERORIENTASI OBJEK



FEMBIANA DIKA SAPUTRI

103132400026

PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pemrograman Berorientasi Objek (Object Oriented Programming/OOP) merupakan metode pemrograman yang mengelompokkan data dan fungsi ke dalam objek sehingga objek dapat saling berinteraksi. Konsep utama OOP meliputi abstraksi, enkapsulasi, inheritance, dan polymorphism. Abstraksi digunakan untuk mengambil karakteristik dan perilaku penting dari suatu objek, sedangkan enkapsulasi menggabungkan data dengan method serta membatasi akses terhadap data tersebut. Inheritance memungkinkan suatu kelas mewarisi atribut dan method dari kelas lain, sedangkan polymorphism memungkinkan satu interface digunakan untuk menghasilkan aksi yang berbeda.

Dalam Java, class merupakan cetak biru atau rancangan dari objek, sedangkan object merupakan hasil instansiasi dari suatu class. Class dapat memiliki atribut atau field untuk menyimpan data dan method untuk mendefinisikan perilaku objek. Method dapat mengembalikan nilai maupun tidak mengembalikan nilai dengan menggunakan `void`.

Java juga memiliki keyword `this` yang digunakan untuk merujuk pada objek yang sedang digunakan. Keyword ini dapat digunakan ketika nama parameter method sama dengan nama atribut pada class, serta dapat digunakan untuk memanggil constructor lain dalam class yang sama.

B. Guided

1. Kucing

```
package guided;

public class Kucing {
    String nama;
    String ras;

    void bersuara() {
        System.out.println(nama + " " + ras + " meow");
    }
}
```

Output :

-

2. Abstraksi

```
package guided;

public class Abstraksi {
    public static void main(String[] args) {
        Kucing aren = new Kucing();
    }
}
```

```
        aren.nama = "Mujaer";
        aren.ras = "Aren Gentong";
        aren.bersuara();
    }
}
```

Output :

```
Mujaer Aren Gentong meow
```

3. Rekening

```
package guided;

public class Rekening {
    private int saldo = 0;

    public void tambahSaldo(int jumlah) {
        saldo = saldo + jumlah;
        System.out.println("Saldo berhasil ditambahkan.");
    }

    public void tampilkanSaldo() {
        System.out.println("Saldo saat ini: " + saldo);
    }
}
```

Output :

-

4. Enkapsulasi

```
package guided;

public class Enkapsulasi {
    public static void main(String[] args) {
        Rekening rek = new Rekening();

        rek.tambahSaldo(20000000);
        rek.tampilkanSaldo();
    }
}
```

Output :

```
Saldo berhasil ditambahkan.
Saldo saat ini: 20000000
```

5. circle01

```
package guided;

public class circle01 {
    public static final double PI = 3.14159;

    public static double radiansToDegrees(double radians) {
        return radians * (180.0 / PI);
    }

    public double r;

    public double area(){
        return PI * r * r;
    }

    public double circumference(){
        return 2 * PI * r;
    }
}
```

Output :

-

C. Unguided

1. Soal Unguided 1

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah.

Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun – bukan sesuatu yang berbeda-beda tiap kali data dibuat – sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini – memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

Jawaban :

PengolahSuhu.java

```
package unguided;

class PengolahSuhu {
    public static final double NILAI_KOSONG = -1.0;

    private double[] suhuHarian;

    public PengolahSuhu(double[] suhuHarian) {
        this.suhuHarian = suhuHarian;
    }

    public void tampilkanData() {
        for (int i = 0; i < suhuHarian.length; i++) {
            if (suhuHarian[i] == NILAI_KOSONG) {
                System.out.println("Hari " + (i + 1) + " : (kosong)");
            } else {
                System.out.println("Hari " + (i + 1) + " : "
                    + suhuHarian[i] + "°C");
            }
        }
    }

    public int cariIndexKosong() {
        for (int i = 0; i < suhuHarian.length; i++) {
            if (suhuHarian[i] == NILAI_KOSONG) {
                return i;
            }
        }
        return -1;
    }

    public void isiDataKosong() {
        int index = cariIndexKosong();

        if (index != -1) {
            suhuHarian[index] =
                (suhuHarian[index - 1] + suhuHarian[index + 1]) / 2;
        }
    }

    public double hitungRataRata() {
        double total = 0;

        for (int i = 0; i < suhuHarian.length; i++) {
            total += suhuHarian[i];
        }

        return total / suhuHarian.length;
    }
}
```

```
}
```

Main.java

```
package unguided;

public class Main {
    public static void main(String[] args) {

        double[] suhuHarian = {
            30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9
        };

        PengolahSuhu suhu = new PengolahSuhu(suhuHarian);

        System.out.println("=== Data Suhu Awal ===");
        suhu.tampilkanData();

        int indexKosong = suhu.cariIndexKosong();

        System.out.println("\nIndex hari kosong (dimulai dari 0): "
            + indexKosong);

        suhu.isiDataKosong();

        System.out.println("\n=== Data Suhu Setelah Pengisian ===");
        suhu.tampilkanData();

        System.out.printf("\nRata-rata : %.2f°C\n",
            suhu.hitungRataRata());

        System.out.println("\nIsi array suhuHarian di main "
            + "setelah isiDataKosong() dijalankan:");

        for (double nilai : suhuHarian) {
            System.out.print(nilai + " ");
        }

        /*
        * Array di main ikut berubah karena object PengolahSuhu
        * menyimpan referensi ke array yang sama.
        * Jadi ketika isiDataKosong() mengubah data melalui object,
        * perubahan tersebut juga terlihat pada array di main.
        */
    }
}
```

Output :

```
=== Data Suhu Awal ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Rata-rata : 29.39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
30.4 24.3 26.8 29.1 31.4 30.8 32.9 BUILD SUCCESSFUL (total time: 0 seconds)
```

Penjelasan :

Program tersebut menggunakan class PengolahSuhu untuk menyimpan dan mengolah data suhu harian. NILAI_KOSONG digunakan sebagai class field karena nilainya tetap dan digunakan bersama oleh seluruh object, sedangkan suhuHarian merupakan instance field yang menyimpan data suhu masing-masing object. Method cariIndexKosong() digunakan untuk mencari posisi data yang bernilai kosong, kemudian isiDataKosong() mengisi data tersebut dengan rata-rata suhu pada hari sebelum dan sesudahnya. Setelah data lengkap, method hitungRataRata() digunakan untuk menghitung rata-rata seluruh suhu. Array yang berada di main ikut berubah setelah proses pengisian karena constructor menyimpan referensi ke array yang sama, sehingga perubahan yang dilakukan melalui object PengolahSuhu juga terlihat pada array di main.

D. Kesimpulan

Praktikum Modul 3 memberikan pemahaman mengenai konsep dasar Pemrograman Berorientasi Objek pada Java. Melalui praktikum ini, dapat dipahami penggunaan class, object, field, method, dan keyword this, serta konsep dasar OOP seperti abstraksi, enkapsulasi, inheritance, dan polymorphism. Konsep tersebut membantu membuat program menjadi lebih terstruktur dengan mengelompokkan data dan fungsi ke dalam objek yang dapat saling berinteraksi.